

선형대수학

남궁명수

선형대수학은 벡터와 선형 구조를 다루는 수학의 언어로, 수많은 대상들을 “방향·크기·관계”라는 공통된 틀로 표현하고 분석하는 학문이다. 점·선·평면 같은 기하적 대상에서 출발해, 벡터공간·행렬·선형변환·내적·고유값 같은 개념으로 확장되며, 복잡한 문제를 공간에서의 변환과 투영으로 이해하게 해준다. 핵심은 “선형성”이라는 단순한 규칙(더하고, 스케일해도 구조가 유지됨)을 통해, 고차원 데이터나 복잡한 시스템을 조합·분해·압축할 수 있게 만드는 데 있다. 그래서 선형대수학은 단순한 계산 기법이 아니라, 물리·컴퓨터공학·머신러닝·통계·그래픽스 등에서 현상을 구조적으로 이해하고 설계하기 위한 사고 도구로 작동한다.

다음은 선형대수학에서 자주 등장하는 기본 용어들의 핵심 의미를 간단하지만 개념 중심으로 정리한 것이다.

스칼라(scalar): 크기만을 가지는 수로, 벡터나 행렬을 늘리거나 줄이는 계수로 작용한다.

벡터(vector): 크기와 방향을 함께 가지는 수학적 객체로, 공간 속의 점, 이동, 상태, 특성을 표현하는 기본 단위이다.

벡터 공간(vector space): 벡터들이 덧셈과 스칼라 곱에 대해 닫혀 있는 공간으로, 선형대수학에서 모든 논의가 이루어지는 무대이다.

기저(basis): 벡터공간을 생성하는 최소한의 독립적인 방향들의 집합으로, 좌표계의 기준이 된다.

차원(dimension): 기저의 개수로, 공간이 가지는 자유도의 수를 의미한다.

내적(inner product): 두 벡터의 방향 유사도와 크기 정보를 함께 담은 연산으로, 각도·정사영·유사도 판단의 핵심도구이다.

노름(norm): 벡터의 크기를 재는 척도로, 거리와 에너지 개념으로 해석된다.

행렬(matrix): 벡터를 다른 벡터로 보내는 선형변환을 수로 표현한 것으로, 공간의 회전·확대·축소·투영을 기술한다.

선형변환(linear transformation): 벡터공간의 구조를 보존하며 벡터를 변형하는 규칙으로, 행렬은 그 표현이다.

정사영(projection): 벡터를 특정 방향이나 부분공간으로 눌러 그 성분만 남기는 연산이다.

스칼라와 벡터 연산

[1] 스칼라와 벡터

- 스칼라(scalar)
 - 크기만을 가지는 값
 - 벡터의 크기를 조절하는 역할
- 벡터(vector)
 - 크기와 방향을 가지는 수학적·물리적 객체
 - 이동, 힘, 속도 등을 표현

[2] 위치와 n-튜플

- x 만큼 이동하고, y 만큼 이동한 지점은 n-튜플(x, y)로 표현
- 이는 좌표이면서 동시에 위치 벡터(position vector)로 해석 가능

[3] 벡터의 기본 정의

- 벡터는 크기와 방향을 가지는 물리량
- 대수적으로는 성분의 집합, 기하적으로는 공간 속 화살표로 이해한다.

[4] 표준 기저 벡터

- e_1 벡터
 - 첫 번째 성분이 1, 나머지는 0
 - (1, 0)
- e_2 벡터
 - 두 번째 성분이 1, 나머지는 0
 - (0, 1)
- 모든 2 차원 벡터는 $(x, y) = xe_1 + ye_2$ 로 표현할 수 있다.

[5] 벡터의 덧셈

- 벡터의 덧셈은 각 성분끼리 더하는 연산이다.
$$(x_1, y_1) + (x_2, y_2) = (x_1 + x_2, y_1 + y_2)$$
- 기하적으로는 벡터를 이어 붙이는 연산이다.

[6] 벡터의 곱셈

- 벡터는 크기와 방향을 가지므로, 곱셈의 결과가 여러 가지로 정의된다.
- 내적(dot product)
 - 기호: $\cdot$
 - 결과: 스칼라
 - 의미: 두 벡터의 방향 유사도
 - 크기 정보만 남는다.
- 외적(cross product)
 - 기호: $\times$
 - 결과: 벡터
 - 의미: 두 벡터가 만드는 평면에 수직인 방향
 - 크기와 방향을 모두 가진다.
- 다이애딕 곱(dyad, outer product)
 - 기호: $\otimes$
 - 결과: 행렬(2 차 텐서)
 - 의미: 한 방향을 다른 방향으로 보내는 변환
 - 크기와 두 개의 방향을 가진다.

[7] 벡터 노름(vector norm)

- 벡터의 크기(길이)를 의미한다.

$$\|v\| = \sqrt{v_1^2 + v_2^2}$$

- 모든 성분의 제곱을 더한 후 제곱근을 취한다.

[8] 위치 벡터(located vector)

- 시작점이 함께 주어진 벡터
- 시작점은 벡터의 성분이 아니라 기하적 정보이다.
- 선형대수에서는 보통 시작점을 원점으로 옮겨 다룬다.

[9] 벡터의 정의 관점

- 대수적(algebraic) 정의 : 성분과 연산을 통해 벡터를 이해
- 기하적(graphical) 정의 : 방향과 크기를 통해 벡터를 이해

[10] 벡터를 그림으로 이해하는 이유

- 3 차원 공간에서 서로 독립인 벡터 a, b, c 를 그리고 덧셈, 내적, 외적 등의 결과를 그림으로 표현하면 연산의 의미를 직관적으로 이해할 수 있다.

[11] 수직 조건

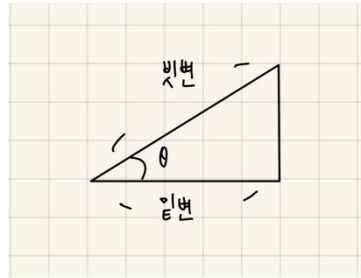
$$a \cdot b = 0$$

이면 두 벡터는 서로 수직이다. (단, a, b 는 0 이 아니어야 한다)

코사인(cosine)

기존의 코사인에 대한 이해는

$$\cos \theta = \frac{\text{밑변}}{\text{빗변}}$$



이라는 공식에 머물러 있었다.

하지만 벡터를 도입하면서 코사인은 전체 길이 중에서 특정 방향(가로 방향)으로 실제 쓰인 비율을 나타내는 값으로 해석할 수 있다.

여기서 진짜 중요한 것은 ‘각도 그 자체’가 아니라, 얼마나 가로 방향으로 쓰였는지 이다.

직관적 이해

길이가 1 인 막대가 있다고 가정한다.

- 막대를 완전히 가로로 두면 : 가로 길이 = 1
- 막대를 조금 기울이면 : 가로 길이 < 1
- 막대를 완전히 수직으로 세우면 : 가로 길이 = 0

즉, 막대의 전체 길이는 같아도 가로 방향으로 쓰인 길이는 방향에 따라 달라진다.

코사인의 의미

빗변의 길이를 1 로 고정했을 때,

가로 방향(밑변)이 얼마나 쓰였는지를 나타내는 값이 바로 코사인이다.

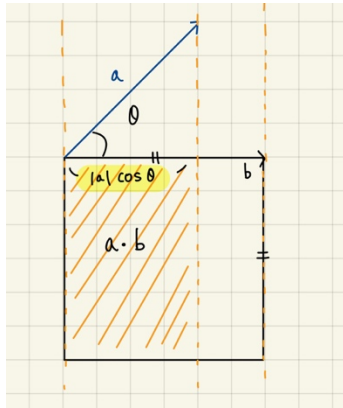
$\cos 0^\circ = 1$ (완전히 가로)

$\cos 90^\circ = 0$ (완전히 수직)

따라서, 코사인은 각도의 이름이 아니라, 전체 중에서 ‘특정 방향으로 쓰인 비율’을 나타내는 값이다. 이 해석을 받아들이면 뒤에 설명하겠지만 내적, 정사영, 벡터 분해가 모두 자연스럽게 연결된다.

- 내적 = 코사인을 계산하는 연산
- 정사영 = 코사인을 벡터로 만드는 것

dot product(내적)



$$a \cdot b$$

- “벡터 a가 벡터 b 방향으로 얼마나 강하게 겹쳐 있는가”를 스칼라 하나로 요약한 값이다.

그래픽적으로 왜 ‘직사각형 면적’처럼 보이나

$$a \cdot b = ab \cos \theta$$

- $|a| \cos \theta$ (밑변 = 빗변 $\cos \theta$)

- a를 b 방향으로 정사영한 길이

- $|b|$

- 그 방향의 기준 길이

- 그림으로 보면 (정사영된 길이) × (기준 길이)

- 벡터의 내적에서 사이각이 커질수록 면적이 줄어들게 된다.

- $a \perp b$ 에 가까워질수록 면적이 줄기에 방향이 다르다는 것을 파악 가능

- $a \parallel b$ 에 가까워질수록 방향이 비슷해진다는 것을 파악 가능

- 면적의 의미는 “한 벡터가 다른 벡터 기준에서 얼마나 강하게 작용하는가”를 구하는 것이다.

$$\Rightarrow a \cdot b = |a||b| \cos \theta$$

따라서, a가 b의 방향에 대해 얼마나 같은 방향 성분을 가지고 있는지를 측정한 값이다.

- b를 기준으로 삼았을 때

- a가 그 방향으로 얼마나 밀고 있는가를 숫자로 만든 것

- 내적 값의 성질

- $a \cdot b > 0$: a가 b 방향으로 기여

- $a \cdot b = 0$: a가 b 방향으로 아무 영향 없음

- $a \cdot b < 0$: a가 b 방향을 방해(반대)

정사영(projection)이란

어떤 대상을 특정 방향 방향으로만 표현하도록 강제했을 때 남는 결과를 말한다. 벡터는 원래 자유롭게 공간에 놓일 수 있지만, 기준이 되는 방향이 주어지면 그 방향으로만 자신을 드러낼 수 있다. 이때 원래 벡터에서 기준 방향과 나란한 성분만 남고, 나머지 성분은 모두 제거된다. 이렇게 기준 방향 위에 “가장 자연스럽게 눌려진 흔적”이 바로 정사영이다. 핵심은 벡터를 바꾸는 것이 아니라, 관점을 제한하는 것이다.

정사영이 “가장 자연스럽다”고 말하는 이유는, 그것이 왜곡 없이 가장 가까운 표현이기 때문이다. 기준 방향으로 표현할 수 있는 모든 벡터 중에서, 원래 벡터와의 차이가 가장 작은 것이 정사영이다. 그래서 정사영 이후에 남는 차이(오차)는 항상 기준 방향과 수직이 된다. 이 성질 때문에 정사영은 근사, 최적화, 최소 오차 문제의 중심 개념이 된다.

dot product(내적)는 바로 이 정사영이 얼마나 되었는지를 수치로 측정하는 도구이다. 닷은 벡터 두개가 서로 얼마나 같은 방향을 보고 있는지 알려준다. 완전히 같은 방향이면 최대, 서로 수직이면 0, 반대 방향이면 음수가 된다. 즉, 닷은 길이를 재는 연산이 아니라 방향의 일치도를 재는 연산이다. 정사영이 “그림자”라면, 닷은 “그 그림자가 얼마나 진한지”를 말해준다.

그래서 이론적으로 보면, 정사영은 기하적 개념이고 닷은 그 개념을 수로 표현한 것이다. 정사영은 “무엇이 남는가”에 대한 답이고, 닷은 “얼마나 남았는가”에 대한 답이다. 이 둘은 계산 이전에 이미 같은 이야기를 하고 있으며, 선형대수는 이 관계를 다양한 형태로 확장해 나가는 학문이다.

sin (사인) – 맞보기

$$\sin \theta = \frac{\text{높이}}{\text{빗변}}$$

- 전체 길이 중에서 세로 방향(수직방향)으로 쓰인 비율을 의미한다.

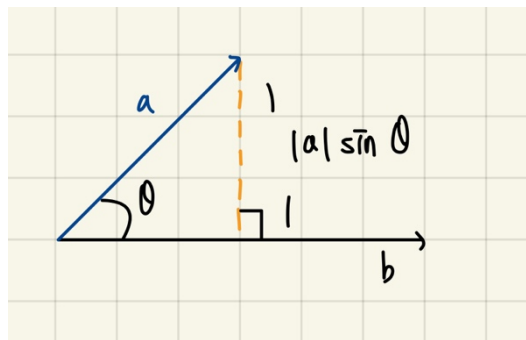
직관적 이해

길이가 1 인 막대를 기준으로 생각해보면

- 막대를 완전히 가로로 두면 : 세로 성분 = 0
- 막대를 조금 기울이면 : 세로 성분 > 0
- 막대를 완전히 수직으로 세우면 : 세로 성분 = 1

즉, 사인은 막대가 세로 방향으로 얼마나 기여하고 있는지를 나타낸다.

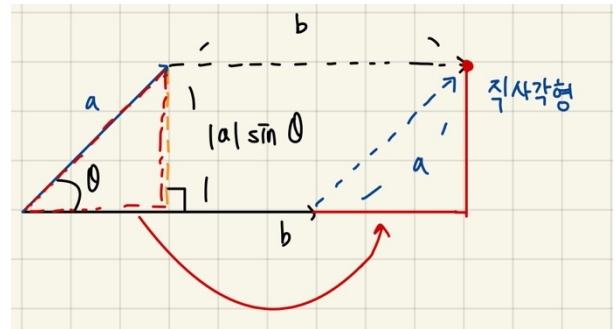
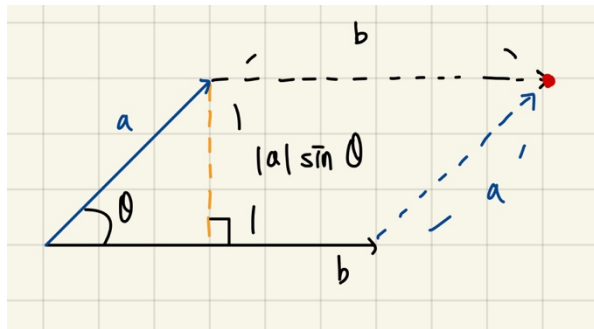
$$\Rightarrow \text{높이} = \text{빗변} \sin \theta$$



이는 뒤이어 평행사변형의 면적으로 이어진다.

사인(sin)은 벡터의 길이 자체를 재는 함수가 아니라, 두 방향이 얼마나 서로 어긋나 있는지를 드러내는 값이다. 한 벡터를 다른 벡터를 기준으로 바라볼 때, 사인은 그 벡터가 기준 방향에 대해 얼마나 수직으로 분리되어 있는가를 나타낸다. 이 수직 성분은 단순한 길이가 아니라, 두 벡터가 함께 만들어내는 공간의 넓이와 직접적으로 연결된다. 왜냐하면 넓이는 “같은 방향으로 겹친 정도”가 아니라, 서로 독립적인 두 방향이 만들어내는 퍼짐에서 발생하기 때문이다. 따라서 두 벡터가 완전히 평행하면 사인은 0 이 되어 넓이가 사라지고, 두 벡터가 서로 수직일수록 사인은 커져 넓이가 최대가 된다. 이런 이유로 사인은 자연스럽게 “분리 정도”를 측정하는 역할을 하게 되고, 그 분리의 기하적 결과가 바로 평행사변형의 면적으로 나타난다.

평행사변형의 면적



평행사변형의 면적을 구하는 공식은 $|a||b|\sin\theta$ 이다.

따라서, 두 벡터가 서로 다른 방향으로 공간을 얼마나 펼치고 있는지를 정량적으로 나타낸 결과라고 볼 수 있다.

벡터 연산과 활용 사례

[1] dot product(내적)

머신러닝에서 내적은 “입력 데이터가 어떤 기준(모델, 가중치) 방향으로 얼마나 잘 맞는지”를 측정하는 도구로 쓰인다.

- 의미: 벡터의 정렬 유도, 유사도, 중요도, 활성화
- 활용 분야:
 - 선형 모델의 예측
 - 로지스틱 회귀, 분류기
 - 코사인 유사도 계산
 - 신경망 내부 활성화
 - 어텐션 메커니즘
 - 손실 함수 최적화

즉, 내적은 정보가 얼마나 기준과 일치하는지를 수치로 나타내는 핵심 연산이다.

[2] 사인(sin)과 외적(cross product)

사인과 외적은 방향과 면적을 다루는 연산으로, 정렬이 아닌 독립성이나 공간적 분리를 측정한다.

- 의미: 방향, 면적, 회전, 거리, 법선
- 활용 분야:
 - 컴퓨터 그래픽스, 3D 모델링
 - 로봇틱스, 자율주행
 - 거리 계산
 - 머신러닝에서 기하학적 제약(loss)
 - 물리 시뮬레이션

외적의 크기는 평행사변형의 면적으로 해석되며, 벡터가 만들어내는 공간적 퍼짐과 관련된다. 방향까지 고려하면, 법선 벡터나 회전 방향 정보도 얻을 수 있다.

unit vector (단위 벡터)

단위 벡터는 길이가 1 인 벡터로, 오직 방향만 나타내는 벡터이다.

벡터 $p = (p_1, p_2)$ 가 있을 때, 단위 벡터 u 는 다음과 같이 정의된다.

$$u = \left(\frac{p_1}{|p|}, \frac{p_2}{|p|}\right) \text{ 또는 } u_p = \frac{p}{|p|}$$

길이를 계산해보면:

$$|u| = \sqrt{\left(\frac{p_1}{|p|}\right)^2 + \left(\frac{p_2}{|p|}\right)^2} = \frac{\sqrt{p_1^2 + p_2^2}}{|p|} = \frac{|p|}{|p|} = 1$$

즉, 길이는 항상 1 이 되며, 방향만 남는다.

단위 벡터는 projection, 내적, 외적 등 벡터 연산에서 기준 방향을 설정할 때 자주 사용된다.

[1] 단위 벡터: 길이와 방향 분리

단위 벡터 $u_p = \frac{p}{|p|}$

- 길이 = 1 (항상)
- 방향 = 원래 벡터 p 가 가리키는 방향

즉, 크기는 제거하고 방향만 남긴 벡터라고 이해하면 된다.

[2] 직관적 예시

벡터 $p = (3, 4)$

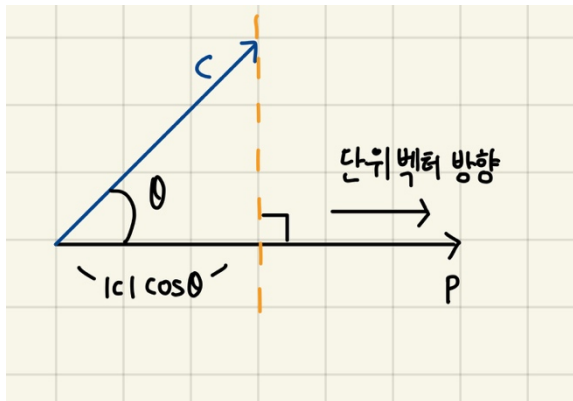
1. 길이 : $|p| = \sqrt{3^2 + 4^2} = 5$

2. 단위 벡터: $\hat{p} = \left(\frac{3}{5}, \frac{4}{5}\right)$

- 길이 = 1
- 방향 = 원래 벡터가 향하던(3, 4) 방향 그대로
- 그래서 방향을 확인하고 싶으면 각 성분의 비율을 보면 된다:

$$\frac{p_2}{p_1} = \frac{4}{3} \text{ (단위벡터도 같은 방향)}$$

projection tensor (정사영 텐서)



벡터를 특정 방향으로 쏙 뽑아내는 연산을 일반화한 것이다.
핵심은 단위 벡터를 기준으로 정사영(projection)하는 것이다.

단위 벡터와 정사영

벡터 c 를 p 방향으로 정사영할 때

- $|c| \cos \theta$: 기준 방향으로 투영된 길이(밑변, 크기)
- $\frac{p}{|p|}$: 방향 단위 벡터

이를 내적(dot product)으로 표현하면

$$|c| \cos \theta \frac{p}{|p|} = \frac{|c||p| \cos \theta}{|p|} \frac{p}{|p|} = \frac{c \cdot p}{|p|} \frac{p}{|p|} = c \cdot \frac{pp}{p \cdot p}$$

여기서 $|p|^2$ 를 $p \cdot p$ 를 이용해 분모를 벡터 내적으로 바꿀 수 있다.

$$a \cdot b = |a||b| \cos \theta$$

벡터 c 를 p 방향으로 projection 하면

$$proj_p c = c \cdot \frac{pp}{p \cdot p}$$

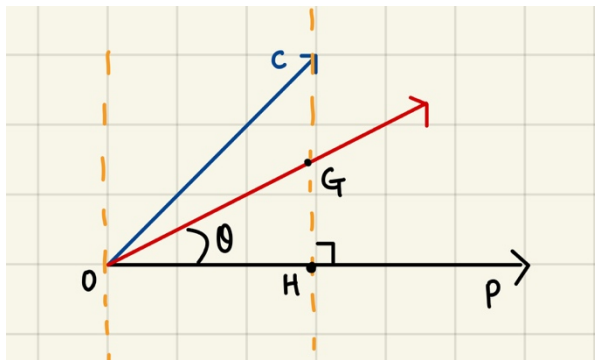
- 의미: c 벡터가 p 방향으로 얼마만큼 기여했는지
- 결과: p 방향으로 놓인 벡터(길이만 남음)

multi-projection(다중 정사영)

벡터는 길이와 방향을 가진다. 우리가 특정 기준 방향, 예를 들어 x 축, y 축, z 축이나 임의의 기준 벡터에 관심이 있을 때, 벡터를 그 기준 방향으로 “썩 뽑아 보는 것”이 바로 projection(정사영)이다. 정사영을 통해 벡터가 특정 방향으로 얼마나 기여하는지를 직관적으로 파악할 수 있다.

하지만 벡터가 한 방향만이 아니라 여러 방향에 영향을 미칠 때는, 단일 방향으로만 projection 을 해서는 벡터가 가지고 있는 전체 정보를 반영할 수 없다. 따라서 여러 기준 방향으로 동시에 projection 하는 과정이 필요하며, 이것이 바로 multi-projection(다중 정사영)이다.

직관적인 예로 3차원 공간에서 벡터 v 를 생각해보자. 기준 방향을 x, y, z 축으로 잡고 각각의 방향으로 그림자를 만든다고 하면, 각 그림자는 벡터가 해당 방향으로 기여하는 정도를 보여준다. 이 세 방향으로 만들어진 그림자를 합치면 원래 벡터 v 를 완전히 복원할 수 있다. 즉, 다중 정사영은 벡터를 여러 독립적인 방향 성분으로 나누고, 각 방향에서의 기여도를 확인하는 과정이라고 이해할 수 있다.



[1] 벡터 정사영(projection) 기본

벡터 c 를 기준 벡터 p 방향으로 썩 뽑는다고 생각한다

1. 길이 계산: 벡터 c 가 p 방향으로 얼마나 기여하는지

$$l = c \cdot \frac{p}{|p|}$$

2. 방향 부여: 단위 벡터 $p = \frac{p}{|p|}$ 를 곱해서 방향까지 포함

3. 내적(dot) 활용

$$c \cdot p = |c||p| \cos \theta \Rightarrow |c| \cos \theta = c \cdot \frac{p}{|p|}$$

[2] 단위 벡터와 방향

- 단위 벡터 $p = \frac{p}{|p|}$: 길이를 1로 만들어 방향만 살림
- 내적에서 나온 밑변 길이에 곱하면 정사영 벡터 완성

$$\overrightarrow{OH} = l \frac{p}{|p|} = c \cdot \frac{pp}{p \cdot p}$$

[3] 다른 방향 부여(OG 벡터 개념)

c 벡터를 p 방향으로 투영한 뒤, a 방향으로 재조정하는 상황을 생각한다.

1. 기존 길이: c 벡터가 p 방향으로 쏙 뽑힌 길이

$$l = c \cdot \frac{p}{|p|}$$

2. 다른 방향으로 맞춤(a 방향)

- p 방향 밑변을 a 방향으로 바꾸려면 $\cos \theta$ 로 길이 보정

$$\overrightarrow{OG} = \frac{l}{\cos \theta} \frac{a}{|a|}$$

- 단위 벡터 $\frac{a}{|a|}$: 방향 부여

3. 내적(dot) 형태로 표현

$$a \cdot p = |a||p| \cos \theta$$

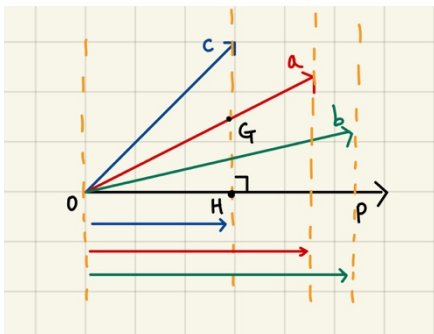
$$\cos \theta = \frac{a \cdot p}{|a||p|}$$

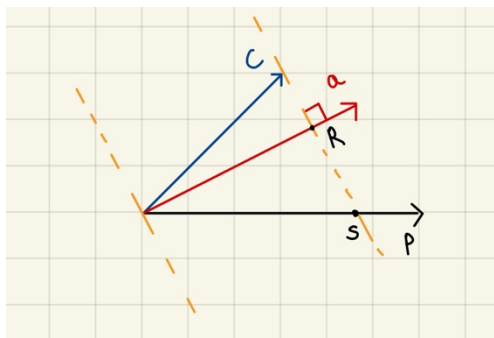
$$\overrightarrow{OG} = c \cdot \frac{p}{|p|} \frac{|a||p|}{a \cdot p} \frac{a}{|a|} = c \cdot \frac{ap}{a \cdot p}$$

- 분자: c 벡터가 p 방향으로 기여한 길이
- 분모: a 와 p 사이 관계 보정
- 결과: c 벡터를 a 방향으로 재배치한 OG 벡터

[4] 직관적 이해

- c $\rightarrow$ p 방향 밑변 길이 l 추출
- p $\rightarrow$ a 방향으로 길이 재조정
- a 방향 단위 벡터 곱 : 방향까지 완성
- 쉽게 말하면 c 가 p 방향으로 얼마나 쓰였는지, c 가 a 방향으로 얼마나 쓰였는지를 찾는 것이다.
- 최종 OG 벡터 = 프로젝션은 p 방향, 벡터 방향은 a 방향(길이와 방향 모두 보정된 벡터)
 - projection = 길이 추출(기준 방향)
 - OG 벡터 = 길이는 추출, 방향은 다른 기준으로 재조정





위 그림은 projection 의 기준을 바꾼 경우이다.

[1] OR 벡터

$$\overrightarrow{OR} = c \cdot \frac{aa}{a \cdot a}$$

- 해석:

1. 기준 방향 = a

2. 벡터 c 를 a 방향으로 투영 → 밑변 길이와 방향 모두 a 에 맞춤

- 말 그대로 c 를 a 방향으로 projection 한 것

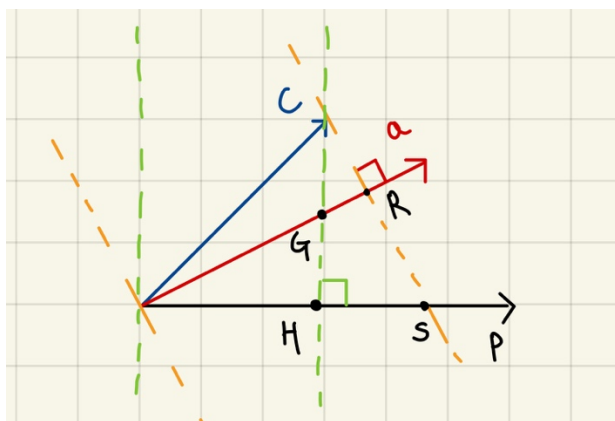
[2] OS 벡터

$$\overrightarrow{OS} = c \cdot \frac{pa}{p \cdot a}$$

- 해석:

1. 밑변은 a 방향으로 계산 → c 가 a 방향으로 기여한 길이

2. 방향은 p 로 재조정 → 방향 p 단위 벡터 사용



위 그림은 두 projection 의 기준이다.

$$\overrightarrow{OH} = c \cdot \frac{pp}{p \cdot p}$$

$$\overrightarrow{OG} = c \cdot \frac{ap}{a \cdot p}$$

$$\overrightarrow{OR} = c \cdot \frac{aa}{a \cdot a}$$

$$\overrightarrow{OS} = c \cdot \frac{pa}{p \cdot a}$$

머신러닝/컴퓨터공학 관점

[1] 특성 분리

- 입력 벡터를 여러 기준 방향으로 분해 → 각 성분별 중요도 분석
- 예: 어텐션 메커니즘, 임베딩 벡터 유사도 계산

[2] 차원 축소 및 투영

- PCA, SVD 같은 차원 축소 기법에서 벡터를 여러 기준 축으로 투영
- 데이터 구조 이해 및 효율적 연산 가능

[3] 정규화 및 방향 보정

- 벡터를 기준 벡터 방향으로 맞춘 뒤, 다른 방향으로 재조정 → 모델 학습 안정화
- 예: 신경망, weight update, feature scaling

[4] 유사도, 상관 관계 계산

- 서로 다른 기준 방향으로 벡터를 projection → 공간 내 관계 파악
- cos similarity, attention score 계산 등에 활용

[5] 물리 기반/기하학적 제약 처리

- 로봇공학, 자율주행, 3D 그래픽스에서 벡터 분해
- 힘, 속도, 법선 벡터 성분 분석 시 유용

Projection 과 벡터 부호의 관계

$$\frac{pa}{p \cdot a} = \frac{(-p)a}{(-p) \cdot a} = \frac{p(-a)}{p \cdot (-a)} = \frac{(-p)(-a)}{(-p) \cdot (-a)}$$

- 벡터 부호를 동시에 바꿔도 projection 결과는 같다
- 이유: projection 공식에서 분자와 분모가 모두 선형적으로 벡터에 비례
- 즉, 방향만 반대인 벡터를 기준으로 해도 정사영 벡터는 같은 선상에 존재

identity tensor (항등 텐서)

텐서는 벡터와 행렬을 일반화한 개념으로, 여러 차원을 가진 데이터를 표현할 수 있는 수학적 도구이다. 벡터가 1 차원 데이터를, 행렬이 2 차원 데이터를 나타낸다면, 텐서는 그 이상의 차원에서도 동일한 구조를 확장하여 표현할 수 있다.

이러한 관점에서, Identity tensor(항등 텐서)는 벡터와 행렬에서의 항등원 개념을 그대로 일반화한 특별한 텐서라고 볼 수 있다. 즉, 텐서 연산에서 Identity tensor 와 결합하면, 다른 텐서의 값을 변경하지 않고 그대로 유지하게 된다.

구체적으로 보면, 1 차원에서는 항등원이 단순히 숫자 1 이며, 2 차원에서는 단위 행렬 I 가 그 역할을 수행한다. 이를 n 차원 텐서로 확장하면, Identity tensor 역시 연산 대상 텐서를 그대로 보존하면서, 텐서 공간 내에서 항등원의 역할을 수행하게 된다. 따라서 Identity tensor 는 텐서 세계에서의 “1” 혹은 “단위 행렬”과 같은 존재로 이해할 수 있으며, 벡터와 행렬에서 익숙한 항등원 개념을 자연스럽게 일반화한 구조적 도구라고 할 수 있다.

[1] 정의

- 2 차원에서 단위 텐서 I 정의:

$$I = ii + jj$$

- 표준 기저 벡터:

$$\begin{aligned} i &= e_1 = (1, 0) \\ j &= e_2 = (0, 1) \end{aligned}$$

- ii 와 jj 는 텐서곱(dyad) 형태

- 행렬 표현:

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

[2] 벡터와 단위 텐서 곱

- 벡터 $a = a_1i + a_2j$ 와 단위 텐서 곱:

$$a \cdot I = (a_1I + a_2I) \cdot (ii + jj)$$

- 전개하면:

$$a_1i \cdot ii + a_1i \cdot jj + a_2j \cdot ii + a_2j \cdot jj$$

[3] 각 항 계산

- $i \cdot ii = i(i \cdot i) = i$ (i 는 단위 벡터 $i \cdot i = 1$)

- $i \cdot jj = i(j \cdot j) = 0$ ($i \perp j \rightarrow i \cdot j = 0$)

- $j \cdot ii = 0$

- $j \cdot jj = j$

$$a \cdot I = a_1i + a_2j = a$$

[4] 결론

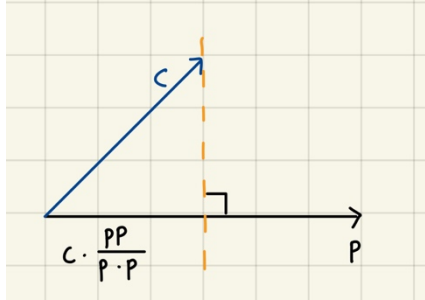
- 단위 텐서는 벡터를 그대로 놔두는 역할

$$a \cdot I = I \cdot a = a$$

- 즉, 변환을 하지 않는 항등 연산자와 같은 개념

complementary projection tensor (보완 정사영 텐서)

그 방향과 수직인 성분을 남기는 역할을 한다. 즉, 기준 벡터를 따라 투영된 성분과 complementary projection tensor 를 통해 얻은 성분을 합치면 원래 벡터를 완전히 복원할 수 있으며 서로 직교 관계에 있다.



$$c - c \cdot \frac{pp}{p \cdot p} = c \cdot I - c \cdot \frac{pp}{p \cdot p} = c(I - \frac{pp}{p \cdot p})$$